

MLOPS · PRODUCTION ML SYSTEM

Minimum Viable Autonomous Self-Healing MLOps System

Automated Model Monitoring, Drift Detection, Retraining and Deployment Pipeline



Python



Scikit-Learn



FastAPI



MLflow



TMDB API



Docker



WHY THIS MATTERS

Problem Statement

Traditional ML models become outdated after deployment because real-world data continuously changes.

- User behaviour changes
- New patterns appear
- Data distribution shifts

THIS CAUSES

- Lower model accuracy
- Wrong predictions
- Manual retraining effort



GOAL

Create an ML system that can:



Monitor new data



Detect changes



Retrain automatically



Update itself safely



APPROACH

Proposed Solution

Autonomous Self-Healing ML Pipeline — the system automatically:

1



Collects new external data

2



Checks for data drift

3



Creates new training data

4



Retrains candidate models

5



Compares performance

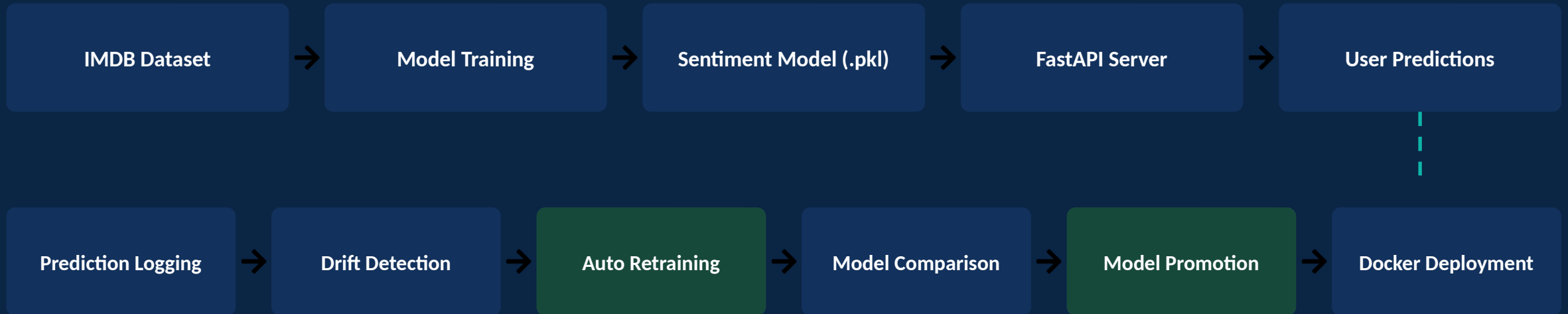
6



Promotes better models



Complete Architecture



Closed-loop feedback: predictions feed drift detection, which triggers retraining and safe promotion.



Dataset Details



Initial Training Dataset

IMDB Movie Review Dataset

CONTAINS

- Movie review text
- Sentiment label

LABELS

Positive

Negative

Used for training the first production model.

EXAMPLE

Review

"The movie was amazing"

Label

Positive



MODEL V1

Initial Model Training

```
src/train.py
```

1 Load IMDB Dataset

2 Text Processing

3 TF-IDF Vectorization

4 Logistic Regression

5 Train Model

6 Save Model

OUTPUT

```
models/sentiment_model.pkl
```

Serialized scikit-learn pipeline (vectorizer + classifier), ready to be loaded by the FastAPI service.

INITIAL ACCURACY

~89%

Baseline production accuracy on held-out IMDB test split, before any live drift or retraining occurs.



SERVING LAYER

API Development

`src/app.py`

Framework: FastAPI

Purpose: expose the trained ML model as a real-time web service that returns sentiment predictions for any review text.



EXAMPLE INPUT

```
"This movie was fantastic"
```

→ *POST /predict*

EXAMPLE OUTPUT

```
{ "prediction": "positive" }
```



Prediction Logging

Folder

logs/

File

predictions.csv

PURPOSE

Store every incoming prediction so production behaviour can be reviewed and monitored over time.

WHY?

Production data is required for monitoring model behaviour and detecting drift later in the pipeline.

STORES

Timestamp

2026-07-09T14:22:10Z

Review

"Great cinematography..."

Prediction

positive



FRESH SIGNAL

External Data Collection

```
collect_reviews.py
```



Data Source: TMDB API

PURPOSE

Collect the latest movie reviews so the model can be evaluated against real, current-day language patterns.

CONTAINS

- Movie ID
- Review text
- Collection timestamp

CREATES

```
external_data/tmdb_reviews.csv
```



Continuously growing pool of real-world, unlabeled movie reviews — the raw material for drift checks and retraining.



Pseudo Labeling



PROBLEM

TMDB reviews do not contain sentiment labels.



SOLUTION

Use the current model to generate labels.



New Review

"This movie is excellent"



Current Model

Runs inference



Pseudo Label

Positive

Only high-confidence predictions are selected — low-confidence pseudo-labels are discarded to avoid reinforcing errors.



Drift Detection

```
drift_detector_v2.py
```

Purpose: find meaningful changes between the reference data and current production data.

Reference Data

VS

Current Data

CHECKS

- 1 Text length difference
- 2 Sentiment distribution shift

OUTPUTS

NO_DRIFT

MODERATE_DRIFT

STRONG_DRIFT



LEARNING LOOP

Retraining Pipeline

```
create_retraining_dataset  
.py
```

```
retrain_candidate.py
```

Original IMDB Data

+

New High-Confidence Data



Combined Dataset



Train Candidate Model

OUTPUT

```
candidate_model.pkl
```



SAFETY GATE

Model Evaluation & Promotion

`model_comparison.py`

`promote_model.py`

Current Model

VS

Candidate Model

DECISION LOGIC



If candidate accuracy is higher

→ Promote Candidate



Otherwise

→ Keep Existing Model

Prevents bad model deployment.



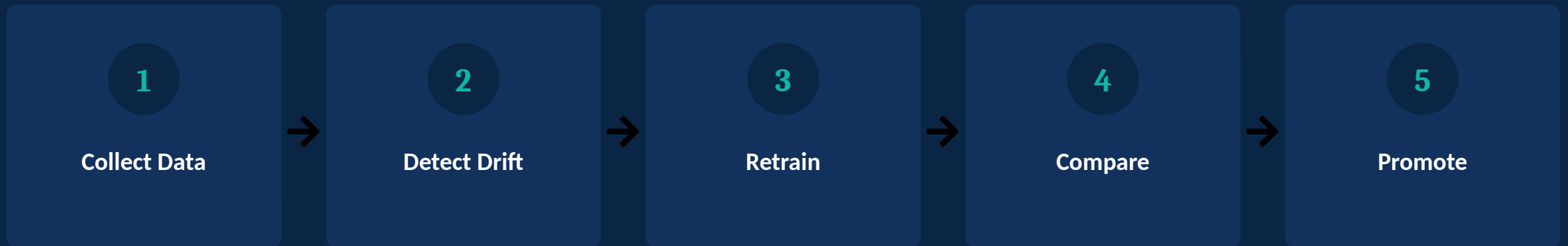
ORCHESTRATION

Self-Healing Automation

```
self_healing_pipeline.py
```

One single command runs the entire loop end-to-end:

```
$ python src/self_healing_pipeline.py
```



Fully automated: from raw external data to a safely promoted model, with zero manual steps.



REPRODUCIBILITY

Experiment Tracking

MLflow Integration

TRACKS



Model versions



Accuracy



Parameters



Experiments

BENEFITS

Reproducibility

Every run's code, data and parameters can be re-created exactly.

Model monitoring

Live visibility into how metrics evolve across retraining cycles.

Experiment comparison

Side-by-side view of candidate vs. production model runs.



PACKAGING

Docker Deployment

Dockerfile

.dockerignore

Purpose: package the complete application — API, model, and dependencies — into one portable container.

BUILD

```
docker build -t self-healing-mlops .
```

RUN

```
docker run -p 8000:8000 self-healing-mlops
```



BENEFIT

Runs anywhere with the same environment

No dependency mismatches between development, staging, and production — the container is the single source of truth for the runtime environment.



DELIVERED

Final Results



Sentiment Prediction API



Automated Data Collection



Drift Detection



Pseudo Labeling



Automatic Retraining



Model Promotion



Docker Deployment



End-to-End MLOps Pipeline

SUMMARY

Conclusion

This project demonstrates a production-style MLOps workflow.

THE SYSTEM CAN

-  **Detect model degradation**
-  **Learn from new data**
-  **Retrain automatically**
-  **Safely update models**



It reduces manual ML maintenance and improves reliability — closing the loop from raw data to a safely deployed, self-correcting model.